



Simula
UiB

Algebraic Attacks on Arithmetization-Oriented Primitives - 2

Morten Øy garden

University of Bergen and Simula UiB

ALPEC Summer School, Trento

September 2026

Resultants

[Yan+24], with elements of [Bar+25; Bak+26] mixed in.



Back to CICO-1

Constrained Input **Constrained Output**

$$P(*, \dots, *, 0) = (*', \dots, *', 0)$$



Elimination Theory

For an ideal $I \subset \mathbb{F}[x, z_1, \dots, z_r]$, we have the i -th *elimination ideal*

$$I_i = I \cap \mathbb{F}[x, z_1, \dots, z_{r-i}].$$



Elimination Theory

For an ideal $I \subset \mathbb{F}[x, z_1, \dots, z_r]$, we have the i -th *elimination ideal*

$$I_i = I \cap \mathbb{F}[x, z_1, \dots, z_{r-i}].$$

The Elimination Theorem

Let G be a Gröbner basis of I w.r.t. the lexicographic order $x < z_1 < \dots < z_r$. Then $G_i = G \cap \mathbb{F}[x, z_1, \dots, z_{r-i}]$ is a Gröbner basis of I_i .



Elimination with Resultants

(Of two Polynomials)

Consider $f, p \in R[x]$ for some commutative ring R , where

$$f = \sum_{i=0}^{\gamma} a_i x^i, \quad a_i \in R, \quad g = \sum_{i=0}^{\delta} b_i x^i, \quad b_i \in R.$$



Elimination with Resultants

(Of two Polynomials)

Consider $f, p \in R[x]$ for some commutative ring R , where

$$f = \sum_{i=0}^{\gamma} a_i x^i, \quad a_i \in R, \quad g = \sum_{i=0}^{\delta} b_i x^i, \quad b_i \in R.$$

The *Sylvester matrix* of f and g in $R^{(\gamma+\delta) \times (\gamma+\delta)}$ is defined as:

$$\text{Syl}_x(f, g) = \left[\begin{array}{cccccc} a_{\gamma} & \cdots & a_1 & a_0 & & 0 \\ & \ddots & & \ddots & \ddots & \\ 0 & & a_{\gamma} & \cdots & a_1 & a_0 \\ b_{\delta} & b_{\delta-1} & \cdots & b_0 & & 0 \\ & \ddots & \ddots & & \ddots & \\ 0 & & b_{\delta} & b_{\delta-1} & \cdots & b_0 \end{array} \right] \left. \begin{array}{l} \left. \begin{array}{l} \left. \begin{array}{l} \left. \begin{array}{l} a_{\gamma} \\ \vdots \\ a_1 \\ a_0 \end{array} \right\} \right\} \delta \\ \left. \begin{array}{l} 0 \\ b_{\delta} \\ b_{\delta-1} \\ \vdots \\ b_0 \end{array} \right\} \gamma \end{array} \right\} \gamma + \delta \end{array} \right\} \cdot$$



Elimination with Resultants

(Of two Polynomials)

The resultant of f and g with respect to x is defined as:

$$\text{Res}_x(f, g) = |Syl_x(f, g)| \in R.$$

$\text{Res}_x(f, g)$ is a polynomial in the coefficients of f and g that does not depend on x .



Recall the Shape of CICO-1 Griffin

$$\begin{aligned}z_1^3 &= f_1(x), \\z_2^3 &= f_2(x, z_1), \\&\vdots \\z_r^3 &= f_r(x, z_1, \dots, z_{r-1}), \\0 &= g(x, z_1, \dots, z_r).\end{aligned}$$

Note the triangular shape: only 2 polynomials depend on z_r ; only 3 polynomials depend on z_{r-1} and so on.



A Succession of (two-polynomial) Resultants

For “our” polynomial systems, we can compute generators for

$$I_1 \supseteq I_2 \supseteq \dots \supseteq I_r = I \cap \mathbb{F}[x],$$

by successively computing the resultants of two polynomials.



A Succession of (two-polynomial) Resultants

For “our” polynomial systems, we can compute generators for

$$I_1 \supseteq I_2 \supseteq \dots \supseteq I_r = I \cap \mathbb{F}[x],$$

by successively computing the resultants of two polynomials.

Write $f_i = z_i^\alpha - p_i$. Then

$$I_1 = \langle f_1, f_2, \dots, f_{r-1}, \operatorname{Res}_{z_r}(g, f_r) \rangle,$$



A Succession of (two-polynomial) Resultants

For “our” polynomial systems, we can compute generators for

$$I_1 \supseteq I_2 \supseteq \dots \supseteq I_r = I \cap \mathbb{F}[x],$$

by successively computing the resultants of two polynomials.

Write $f_i = z_i^\alpha - p_i$. Then

$$I_1 = \langle f_1, f_2, \dots, f_{r-1}, \operatorname{Res}_{z_r}(g, f_r) \rangle,$$

$$I_2 = \langle f_1, f_2, \dots, f_{r-2}, \operatorname{Res}_{z_{r-1}}(\operatorname{Res}_{z_r}(g, f_r), f_{r-1}) \rangle,$$

and so on.



Summary and Complexity

We eventually end up with a univariate polynomial in degree D_I , from which we can easily find a solution.

Works well in conjunction with round skipping.



Summary and Complexity

We eventually end up with a univariate polynomial in degree D_I , from which we can easily find a solution.

Works well in conjunction with round skipping.

Lots of improvements/details to be had in how computations are performed. Concrete complexity depends a lot on this.



Summary and Complexity

We eventually end up with a univariate polynomial in degree D_I , from which we can easily find a solution.

Works well in conjunction with round skipping.

Lots of improvements/details to be had in how computations are performed. Concrete complexity depends a lot on this.

Complexity results get a bit messy, but we can sometimes get below the $\sim D_I^\omega \implies$ can yield better CICO-1 attacks than Freelunch.



Resultants and CICO- k ?

Resultants get really messy for $k \geq 2$, as you no longer get a natural 2-polynomial elimination path.



Resultants and CICO- k ?

Resultants get really messy for $k \geq 2$, as you no longer get a natural 2-polynomial elimination path.

Some works have still played around with $k = 2$. The state-of-the-art is [Bak+26] which gets very nice results on CICO-2 (and further improves the resultant computation of CICO-1).



Resultants and CICO- k ?

Resultants get really messy for $k \geq 2$, as you no longer get a natural 2-polynomial elimination path.

Some works have still played around with $k = 2$. The state-of-the-art is [Bak+26] which gets very nice results on CICO-2 (and further improves the resultant computation of CICO-1).

OP3: How do we extend resultant techniques to CICO- k for $k > 2$?



Resultants and CICO- k ?

Resultants get really messy for $k \geq 2$, as you no longer get a natural 2-polynomial elimination path.

Some works have still played around with $k = 2$. The state-of-the-art is [Bak+26] which gets very nice results on CICO-2 (and further improves the resultant computation of CICO-1).

OP3: How do we extend resultant techniques to CICO- k for $k > 2$?

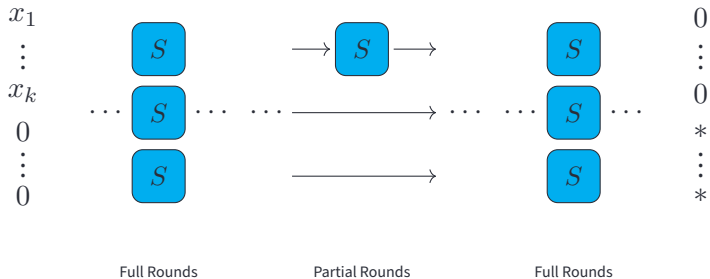
OP4: CICO- k is most relevant for “smaller” field sizes, but we currently do not exploit this. How can we incorporate small field sizes into our resultant attacks? (E.g., guessing variables, field equations,...)



More on CICO- k : Preliminaries



Poseidon and CICO



Recall: (Weighted) Monomial Orders

We define *weighted* monomial orders through a weight vector $w = (w_1, \dots, w_n) \in \mathbb{R}^n$ for the variables $(x_1, \dots, x_n)$.



Recall: (Weighted) Monomial Orders

We define *weighted* monomial orders through a weight vector $w = (w_1, \dots, w_n) \in \mathbb{R}^n$ for the variables $(x_1, \dots, x_n)$.

We can then compare monomials via:

$$\prod_{i=1}^n x_i^{\alpha_i} \implies \sum_{i=1}^n w_i \alpha_i \in \mathbb{R},$$

(ties are determined by other means, e.g., secondary weight vectors.)

Example: Graded orders have $w = (1, \dots, 1)$.



Recall: Regular Systems

For graded monomial orders there is the notion of *regular* polynomial systems, which can be thought of as a higher-degree generalization of linear independence.



Recall: Regular Systems

For graded monomial orders there is the notion of *regular* polynomial systems, which can be thought of as a higher-degree generalization of linear independence.

We know a lot about regular polynomial systems, e.g., the complexity of computing GBs (via the Macaulay bound) and the Bézout bound yields:

$$D_I = \prod_{i=1}^n d_i, \quad d_i = \deg(f_i).$$



Recall: Regular Systems

For graded monomial orders there is the notion of *regular* polynomial systems, which can be thought of as a higher-degree generalization of linear independence.

We know a lot about regular polynomial systems, e.g., the complexity of computing GBs (via the Macaulay bound) and the Bézout bound yields:

$$D_I = \prod_{i=1}^n d_i, \quad d_i = \deg(f_i).$$

Regularity is a generic condition, i.e., most systems have this property, but it does *not* hold for Poseidon.



Weighted Regular Systems

[FEV16]

For a weight vector w , consider the morphism:

$$\begin{aligned} Hom_w : \mathbb{F}[x_1, \dots, x_n] &\longrightarrow \mathbb{F}[x_1, \dots, x_n] \\ f(x_1, \dots, x_n) &\longmapsto f(x_1^{w_1}, \dots, x_n^{w_n}) \end{aligned}$$

Idea: Consider the polynomial system under Hom_w , and things mainly¹ proceed as in the (normal) graded case.

¹Mainly is doing some heavy lifting here. There are further details and conditions to be satisfied: see the paper for more info.



Weighted Regular Systems

[FEV16]

For a weight vector w , consider the morphism:

$$\begin{aligned} Hom_w : \mathbb{F}[x_1, \dots, x_n] &\longrightarrow \mathbb{F}[x_1, \dots, x_n] \\ f(x_1, \dots, x_n) &\longmapsto f(x_1^{w_1}, \dots, x_n^{w_n}) \end{aligned}$$

Idea: Consider the polynomial system under Hom_w , and things mainly¹ proceed as in the (normal) graded case.

In particular, this gives a notion of *weighted regularity*, *weighted Macaulay bound* and *weighted Bézout bound*:

$$D_I \leq \prod_{i=1}^n \frac{\deg_w(f_i)}{w_i}$$

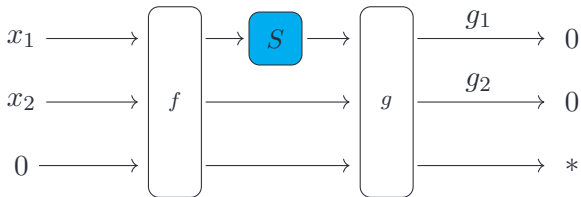
¹Mainly is doing some heavy lifting here. There are further details and conditions to be satisfied: see the paper for more info.



Example: CICO- k for
Poseidon [Bak+25]



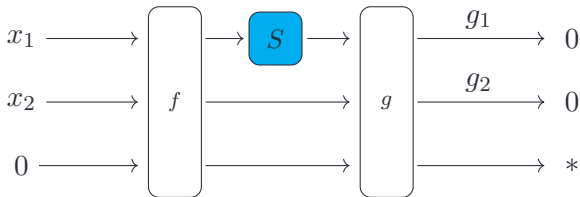
Toy Example 1 - I



Poseidon with a single intermediate round: f and g denotes full rounds. $S : z \mapsto z^\alpha$, $\deg(f) = d_f$, $\deg(g) = d_g$.



Toy Example 1 - I



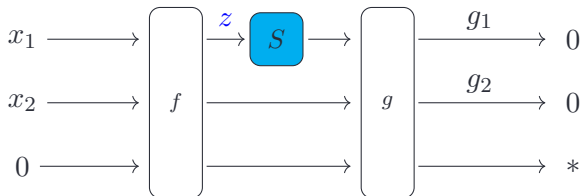
Poseidon with a single intermediate round: f and g denotes full rounds. $S : z \mapsto z^\alpha$, $\deg(f) = d_f$, $\deg(g) = d_g$.

Naively, we can write g_1 and g_2 as functions in x_1, x_2 of degree $\alpha d_f d_g$. The Bezout bound gives us: $D_I \leq (\alpha d_f d_g)^2$,

In practice we observe $D_I = \alpha(d_f d_g)^2$.



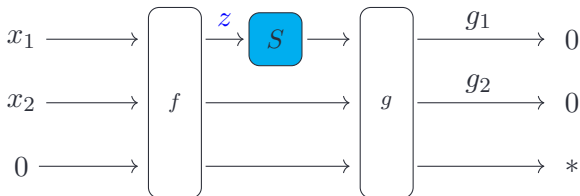
Toy Example 1 - II



Now introduce the variable z and define the polynomial system $Q = (q_1, q_2, q_3) \subset \mathbb{F}[x_1, x_2, z]$ by:



Toy Example 1 - II



Now introduce the variable z and define the polynomial system $Q = (q_1, q_2, q_3) \subset \mathbb{F}[x_1, x_2, z]$ by:

$$q_1 = f_1(x_1, x_2, 0) - z,$$

$$q_2 = g_1(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)),$$

$$q_3 = g_2(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)).$$

(Note: $D_I = D_{\langle Q \rangle}$)



Toy Example 1 - III

$$q_1 = f_1(x_1, x_2, 0) - z,$$

$$q_2 = g_1(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)),$$

$$q_3 = g_2(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)).$$

We define the weight vector $w = (1, 1, \frac{df}{\alpha})$ for the variables (x_1, x_2, z) .



Toy Example 1 - III

$$q_1 = f_1(x_1, x_2, 0) - z,$$

$$q_2 = g_1(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)),$$

$$q_3 = g_2(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)).$$

We define the weight vector $w = (1, 1, \frac{d_f}{\alpha})$ for the variables (x_1, x_2, z) .

$\implies$ All inputs to g will have $\deg_w = d_f$.

Moreover, $\deg_w(q_1) = d_f, \deg_w(q_2) = \deg_w(q_3) = d_f d_g$.



Toy Example 1 - III

$$q_1 = f_1(x_1, x_2, 0) - z,$$

$$q_2 = g_1(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)),$$

$$q_3 = g_2(z^\alpha, f_2(x_1, x_2, 0), f_3(x_1, x_2, 0)).$$

We define the weight vector $w = (1, 1, \frac{d_f}{\alpha})$ for the variables (x_1, x_2, z) .

$\implies$ All inputs to g will have $\deg_w = d_f$.

Moreover, $\deg_w(q_1) = d_f$, $\deg_w(q_2) = \deg_w(q_3) = d_f d_g$.

The (weighted) Bezout bound gives

$$D_I \leq \frac{d_f(d_f d_g)^2}{d_f/\alpha} = \alpha(d_f d_g)^2,$$

which is tight if the system is weighted regular (confirmed by experiments).



More General Cases

Generalizing this toy example to $x_1, \dots, x_k$ (and $\leq k$ outputs) is straightforward using the weighted regularity framework.



More General Cases

Generalizing this toy example to $x_1, \dots, x_k$ (and $\leq k$ outputs) is straightforward using the weighted regularity framework.

Generalizing to $R_p \geq 2$ partial rounds is done as follows: we split the polynomial system into $P \cup Q$, where P is weighted regular in $x_1, \dots, x_k, z$ and Q is made up of “Freelunch-relations” (i.e., coprime leading terms).



More General Cases

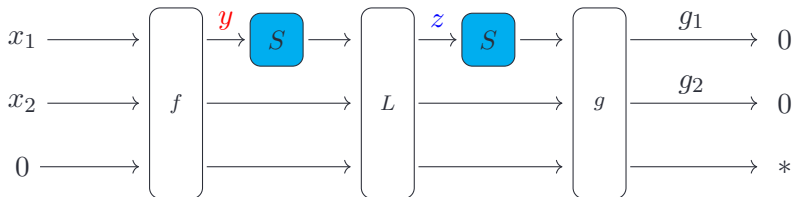
Generalizing this toy example to $x_1, \dots, x_k$ (and $\leq k$ outputs) is straightforward using the weighted regularity framework.

Generalizing to $R_p \geq 2$ partial rounds is done as follows: we split the polynomial system into $P \cup Q$, where P is weighted regular in $x_1, \dots, x_k, z$ and Q is made up of “Freelunch-relations” (i.e., coprime leading terms).

Fact: if Q is weighted regular, then the Gröbner basis $G(P \cup Q) = G(P) \cup G(Q)$. (See paper for details.)



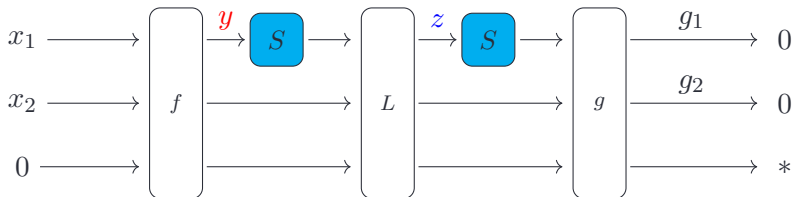
Toy Example 2 - I



We introduce variables y and z and define the polynomial system in $\mathbb{F}[x_1, x_2, y, z]$ by:



Toy Example 2 - I



We introduce variables y and z and define the polynomial system in $\mathbb{F}[x_1, x_2, y, z]$ by:

$$\begin{aligned}q_1 &= f_1(x_1, x_2, 0) - y, \\p &= L(y^\alpha, f_2, f_3) - z, \\q'_2 &= g_1(f_2, f_3, y^\alpha, z^\alpha), \\q'_3 &= g_2(f_2, f_3, y^\alpha, z^\alpha).\end{aligned}$$



Toy Example 2 - II

$$\begin{aligned}q_1 &= f_1(x_1, x_2, 0) - y, & p &= L(y^\alpha, f_2, f_3) - z, \\q'_2 &= g_1(f_2, f_3, y^\alpha, z^\alpha), & q'_3 &= g_2(f_2, f_3, y^\alpha, z^\alpha).\end{aligned}$$

Define the weight $w = (1, 1, \frac{d_f}{\alpha}, \frac{d_f}{\alpha})$ for the variables (x_1, x_2, y, z) , and let $y > x_1, x_2, z$.



Toy Example 2 - II

$$\begin{aligned}q_1 &= f_1(x_1, x_2, 0) - y, & p &= L(y^\alpha, f_2, f_3) - z, \\q'_2 &= g_1(f_2, f_3, y^\alpha, z^\alpha), & q'_3 &= g_2(f_2, f_3, y^\alpha, z^\alpha).\end{aligned}$$

Define the weight $w = (1, 1, \frac{d_f}{\alpha}, \frac{d_f}{\alpha})$ for the variables (x_1, x_2, y, z) , and let $y > x_1, x_2, z$.

$$\implies LM(p) = y^\alpha.$$



Toy Example 2 - II

$$\begin{aligned}q_1 &= f_1(x_1, x_2, 0) - y, & p &= L(y^\alpha, f_2, f_3) - z, \\q'_2 &= g_1(f_2, f_3, y^\alpha, z^\alpha), & q'_3 &= g_2(f_2, f_3, y^\alpha, z^\alpha).\end{aligned}$$

Define the weight $w = (1, 1, \frac{d_f}{\alpha}, \frac{d_f}{\alpha})$ for the variables (x_1, x_2, y, z) , and let $y > x_1, x_2, z$.

$$\implies LM(p) = y^\alpha.$$

Let q_2 be q'_2 reduced by p (i.e., removes the y^α 's from the polynomial). Define q_3 in the same manner.



Toy Example 2 - III

The components of highest (weighted) degree in $Q = (q_1, q_2, q_3)$ will now only depend on x_1, x_2, z , and we can proceed as we did in the previous toy example.



Toy Example 2 - III

The components of highest (weighted) degree in $Q = (q_1, q_2, q_3)$ will now only depend on x_1, x_2, z , and we can proceed as we did in the previous toy example.

Fact: If Q is weighted regular, then the leading monomials in $G(B)$ will only depend on x_1, x_2, z , and hence be coprime with $LM(p) = y^\alpha$.

$$\implies G(Q \cup p) = G(Q) \cup p.$$



Toy Example 2 - III

The components of highest (weighted) degree in $Q = (q_1, q_2, q_3)$ will now only depend on x_1, x_2, z , and we can proceed as we did in the previous toy example.

Fact: If Q is weighted regular, then the leading monomials in $G(B)$ will only depend on x_1, x_2, z , and hence be coprime with $LM(p) = y^\alpha$.

$$\implies G(Q \cup p) = G(Q) \cup p.$$

In particular we have

$$D_I = \frac{d_f^2 (d_f d_g)^2}{(d_f / \alpha)^2} = (\alpha d_f d_g)^2,$$



Summary on CICO— k for Poseidon

- This idea can be generalized nicely to larger R_p .



Summary on CICO— k for Poseidon

- This idea can be generalized nicely to larger R_p .
- There's a bunch of extra optimization and details I'm not mentioning here.



Summary on CICO— k for Poseidon

- This idea can be generalized nicely to larger R_p .
- There's a bunch of extra optimization and details I'm not mentioning here.
- This works very well in conjunction with round-skipping.



Summary on CICO— k for Poseidon

- This idea can be generalized nicely to larger R_p .
- There's a bunch of extra optimization and details I'm not mentioning here.
- This works very well in conjunction with round-skipping.
- The bottleneck is Step 2) (as in FreeLunch), with complexity.

$$\tilde{\mathcal{O}} \left(D_I \left(D_I / \alpha^{R_F} \right)^{\omega-1} \right),$$

field operations.



Summary on CICO— k for Poseidon

- This idea can be generalized nicely to larger R_p .
- There's a bunch of extra optimization and details I'm not mentioning here.
- This works very well in conjunction with round-skipping.
- The bottleneck is Step 2) (as in FreeLunch), with complexity.

$$\tilde{\mathcal{O}} \left(D_I \left(D_I / \alpha^{R_F} \right)^{\omega-1} \right),$$

field operations.

- This generally yields the best known attack on CICO- k for Poseidon, but does not threaten proposed parameters.



CICO- k for Other Schemes

The same ideas work well for CICO- k problems for Neptune [Gra+22] and XHash8 [Ash+26]. ([CR25] also deals with Anemoi CICO- k with different methods).



CICO- k for Other Schemes

The same ideas work well for CICO- k problems for Neptune [Gra+22] and XHash8 [Ash+26]. ([CR25] also deals with Anemoi CICO- k with different methods).

Can be applied to CICO- k for Griffin and ArionHash, but we do not get as tight estimates as we hope for (not weighted regular).

Reasonable CICO- k , $k > 2$, instances for these schemes seems robust.



CICO- k for Other Schemes

The same ideas work well for CICO- k problems for Neptune [Gra+22] and XHash8 [Ash+26]. ([CR25] also deals with Anemoi CICO- k with different methods).

Can be applied to CICO- k for Griffin and ArionHash, but we do not get as tight estimates as we hope for (not weighted regular).

Reasonable CICO- k , $k > 2$, instances for these schemes seems robust.

OP5: Can we extend these techniques for other schemes (XHash12, Rescue [SAD20],...)? Can we improve the estimates for Griffin and ArionHash?



CICO- k for Other Schemes

The same ideas work well for CICO- k problems for Neptune [Gra+22] and XHash8 [Ash+26]. ([CR25] also deals with Anemoi CICO- k with different methods).

Can be applied to CICO- k for Griffin and ArionHash, but we do not get as tight estimates as we hope for (not weighted regular).

Reasonable CICO- k , $k > 2$, instances for these schemes seems robust.

OP5: Can we extend these techniques for other schemes (XHash12, Rescue [SAD20],...)? Can we improve the estimates for Griffin and ArionHash?

OP6: How do we exploit small fields with our Gröbner basis techniques?



Lightning-Round Topics



ZK-Friendly Hashes with LUTs

Some hashes combine Lookup tables (to avoid algebraic attacks) with power maps (to avoid statistical attacks).

Probably a good idea. There's usually no way to model the LUTs in a nice, algebraic manner.

(That doesn't mean things can't go wrong.)

I think there's a general feeling in the community that these schemes have received little cryptanalytic attention (but I personally don't see any fruitful directions here).



MPC-Friendly Encryption

The use-cases in MPC seems mainly to be related to encryption.

Examples: HadesMiMC (keyed version of Poseidon), Ciminion, Hydra.



MPC-Friendly Encryption

The use-cases in MPC seems mainly to be related to encryption.

Examples: HadesMiMC (keyed version of Poseidon), Ciminion, Hydra.

We can draw on the techniques from ZK-friendly hash functions for security analysis, but multiple plaintext-ciphertext pairs gives a type of overdetermined systems we don't see in the Hash setting.



MPC-Friendly Encryption

The use-cases in MPC seems mainly to be related to encryption.

Examples: HadesMiMC (keyed version of Poseidon), Ciminion, Hydra.

We can draw on the techniques from ZK-friendly hash functions for security analysis, but multiple plaintext-ciphertext pairs gives a type of overdetermined systems we don't see in the Hash setting.

OP7: How can we use exploit multiple plaintext-ciphertext pairs in attacks? How do we derive good security arguments in this setting?



Other Families

FHE-Friendly Encryption (HHE): Many subfamilies exists. For the Rasta-family [Dob+18] we typically have very large state (number of branches) which yields nice algebraic security arguments (linearization).



Other Families

FHE-Friendly Encryption (HHE): Many subfamilies exists. For the Rasta-family [Dob+18] we typically have very large state (number of branches) which yields nice algebraic security arguments (linearization).

AO-Encryption for PQ-Signatures: Ciphers specialized for MPC-itH-based signatures. E.g., Rain [Dob+22] and AIM [Kim+23]. Lots of attacks here! (But a story for another time.)

Many other variants exists (as Léo mentioned).



Summary



High-Level Picture - 1

Generally speaking, all AO primitives still require a lot of cryptanalysis, but the maturity varies widely within the different (sub-)families.



High-Level Picture - 1

Generally speaking, all AO primitives still require a lot of cryptanalysis, but the maturity varies widely within the different (sub-)families.

ZK-friendly hash functions have received the most cryptanalytic attention and improvements.



High-Level Picture - 1

Generally speaking, all AO primitives still require a lot of cryptanalysis, but the maturity varies widely within the different (sub-)families.

ZK-friendly hash functions have received the most cryptanalytic attention and improvements.

We've seen some variants break, but things get patched and we've learned a lot more about properly choosing parameters. Things generally seem more stable for newer updates/proposals.



High-Level Picture - 1

Generally speaking, all AO primitives still require a lot of cryptanalysis, but the maturity varies widely within the different (sub-)families.

ZK-friendly hash functions have received the most cryptanalytic attention and improvements.

We've seen some variants break, but things get patched and we've learned a lot more about properly choosing parameters. Things generally seem more stable for newer updates/proposals.

(some caveats exits, e.g., round-skipping, though the most efficient variants here applies to very 'stretched' parameters.)



High-Level Picture - 2

We can distinguish two interesting cases for ZK-friendly hash functions:

- i)* large fields $\gtrsim 256$ bits and CICO-1.
- ii)* fields of size $\ll 128$ bits with CICO- k , $k > 2$.



High-Level Picture - 2

We can distinguish two interesting cases for ZK-friendly hash functions:

- i)* large fields $\gtrsim 256$ bits and CICO-1.
- ii)* fields of size $\ll 128$ bits with CICO- k , $k > 2$.

For case *i)* we are starting to get a fairly good grasp on analysis.



High-Level Picture - 2

We can distinguish two interesting cases for ZK-friendly hash functions:

- i)* large fields $\gtrsim 256$ bits and CICO-1.
- ii)* fields of size $\ll 128$ bits with CICO- k , $k > 2$.

For case *i)* we are starting to get a fairly good grasp on analysis.

There are a lot of unanswered questions for case *ii)*. (That doesn't necessarily mean that things are weak: I rather have the impression parameters are overly conservative in this setting.)



The Open Problems We've Discussed - 1

OP1: Deeper insights into round-skipping.

OP2: Constructing multiplication matrices.

OP3: Improved resultant techniques/analysis for CICO- k , $k \geq 2$.

OP4: Exploiting small fields in resultant attacks ($k \geq 2$).



The Open Problems We've Discussed - 2

OP5: Improved Gröbner basis techniques/analysis for CICO- k , $k \geq 2$.

OP6: Exploiting small fields in Gröbner basis attacks ($k \geq 2$).

OP7: Cryptanalysis of MPC-friendly encryption.

